

Evaluation & Cross-Validation

INFO 521 · student notes · Module 2 · Lecture A · ~6 min read

Module 1 produced a fit; Module 2 asks whether it will hold up on unseen patients. This lecture is the diagnostic half: how to measure whether a model has learned the pattern or merely memorized the sample.

Training error is not generalization error

Basis functions make a model arbitrarily flexible, and flexibility buys fit — a high-degree curve can pass through every training point. But **training error only ever falls with complexity**; a degree- k polynomial can interpolate $k + 1$ points and hit zero training loss. Low training error is cheap and says nothing about the next patient. A zero on the training set may mean a perfect model or a memorized one, and training error alone cannot tell which.

The honest estimate comes from scoring on data the model never saw while fitting. Split patients into a **training** set (fit) and a **test** set (score only); when a knob is also being tuned, a third **validation/dev** set does the tuning so the test set stays sealed. The test set is sacred — touch it once, at the very end.

The U-curve and the generalization gap

Plotting error against complexity, training error slides down while **test error is U-shaped**: high on the left (too simple to catch the signal), lowest in the middle, rising again on the right (fitting noise). The distance between the curves — the **generalization gap** — diagnoses the model: a wide gap with low training error is overfitting; both curves high is underfitting; the bottom of the U is just-right.

Cross-validation

A single train/test split wastes data and is high-variance (a different split gives a different number). **k -fold cross-validation** fixes both: partition into k folds, and in each round one fold validates while the other $k - 1$ train. Every patient validates exactly once and trains $k - 1$ times; the CV error is the average of the round scores. Sweeping complexity produces a stable CV curve whose minimum is the chosen setting.

Selection and reporting are different jobs: **cross-validation selects** the complexity, then you refit on all training data and **report** on the sealed test set. Don't let the data that picks the model also grade it.

Leakage

Cross-validation only tells the truth if folds are honestly separated. Any preprocessing that learns from data — centering, scaling — must be fit *inside each training fold* and applied to the validation fold, never on pooled data. This is the same discipline as Module 1's feature-leakage rule: the answer must not sneak into the inputs (there it was diastolic BP; here it is the fold statistics).

What to remember

- Training error falls with complexity; test/CV error is U-shaped.
- The generalization gap distinguishes overfitting (wide gap) from underfitting (both high).
- k -fold CV gives a stable estimate using all the data; use it to *select*, and a sealed test set to *report*.
- Preprocess inside folds — pooled preprocessing leaks and inflates the estimate.

Through-lines

CV tells you *where* you sit on the U; Lecture B explains *why* the U exists (bias–variance) and gives a knob to move along it (ridge). The leakage rule is the same one introduced in Module 1.